Веса в целевой функции

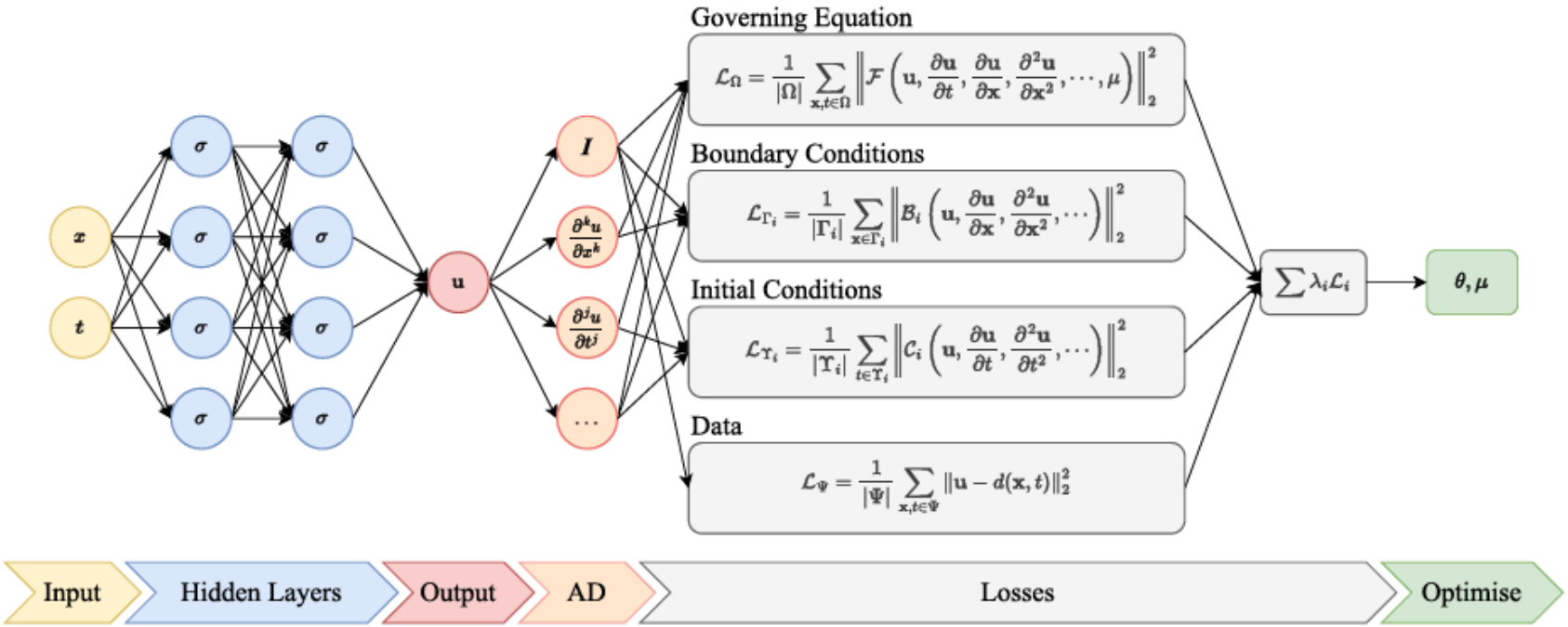
Адаптивные весовые коэффициенты[2]

- Learning Rate Annealing (LRAnnealing)[1], затухание скорости обучения
- GradNorm
- SoftAdapt
- ReLoBRaLo (Relative Loss Balancing with Random Lookback)

Задача PINN–задача многоцелевой оптимизации

$$\mathcal{L}(\theta) = \left(\mathcal{L}_1(\theta), \ldots, \mathcal{L}_k(\theta)\right)^T \tag{1}$$

$$\mathcal{L}(\theta) = \sum_{i=1}^{k} \lambda_i \mathcal{L}_i(\theta), \qquad \lambda_i \in \mathbb{R}_{>0} \tag{2}$$



LRAnnealing

$$\widehat{\lambda}_i(t) = \frac{\max |\nabla_\theta \mathcal{L}_j(\theta)|}{\overline{|\nabla_\theta \mathcal{L}_i(\theta)|}}, \qquad i \in \{1, \ldots, k\}$$

$$\lambda_i(t) = \alpha \lambda_i(t-1) + (1-\alpha)\widehat{\lambda}_i(t)$$

$$\theta^{t+1} = \theta^t - \eta \nabla_\theta \left(\mathcal{L}_c(\theta^t) + \sum_{i=1}^{k} \lambda_i(t) \mathcal{L}_i(\theta^t) \right)$$

где $\overline{|\nabla_\theta \mathcal{L}_i(\theta)|}$ — среднее значение градиента по параметрам θ, а α — гиперпараметр со значением $\alpha = 0.9$, рекомендованным авторами.

GradNorm: балансировка градиентов I

GradNorm адаптивно обучает веса $\lambda_i(t)$ для функций потерь $\mathcal{L}_i$.
Целевая функция для обновления весов:

$$\mathcal{L}(t;\lambda) = \sum_{i=1}^{k} \left| G_\theta^{(i)}(t) - \overline{G}_\theta(t) \times [r_i(t)]^\alpha \right|.$$

Норма градиента для слагаемого i и среднее по всем нормам градиентов:

$$G_\theta^i(t) = \|\nabla_\theta[\lambda_i \mathcal{L}_i(t)]\|_2\,, \qquad \overline{G}(t) = \frac{1}{k}\sum_{i=1}^{k} G_i(t).$$

Относительная скорость обучения задачи:

$$r_i(t) = \frac{\mathcal{L}_i(t)}{\mathcal{L}_i(0)}.$$

GradNorm: балансировка градиентов II

Чем медленнее уменьшается слагаемое, тем больше его относительная скорость $r_i(t)$ по сравнению с остальными. Гиперпараметр α задаёт силу балансировки: при большем α медленно обучающиеся части получают больший вес. **Обновление сети:**

$$\mathcal{L}_{\text{total}}(t) = \sum_{i=1}^{k} \lambda_i(t)\mathcal{L}_i(t), \qquad \theta \leftarrow \theta - \eta\nabla_\theta \mathcal{L}_{\text{total}}(t).$$

GradNorm может быть сформулирована как минимизация:

$$\mathcal{L}(t) = \big(\mathcal{L}(t;\theta), \mathcal{L}(t;\lambda)\big)^T \tag{3}$$

Итог: веса задач обучаются так, чтобы согласовать нормы градиентов с темпом обучения каждой части целевой функции.

GradNorm: балансировка градиентов III

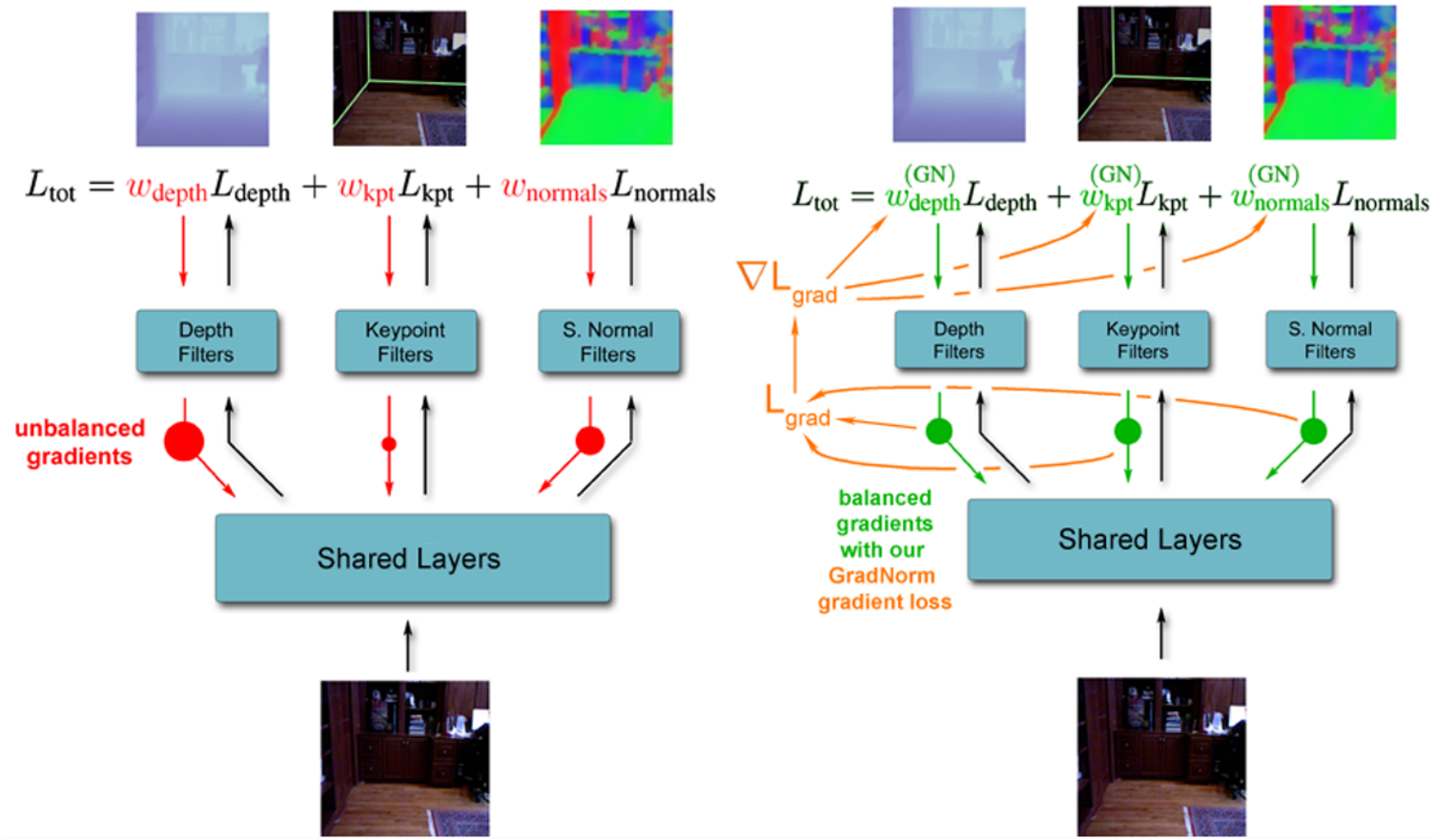


Рис. 1 – Иллюстрация GradNorm из оригинальной статьи [3]

SoftAdapt

Весовые коэффициенты рассчитываются по правилу:

$$\lambda_i(t) = \frac{\exp\big(\mathcal{T}\big(\mathcal{L}_i(t) - \mathcal{L}_i(t-1)\big)\big)}{\sum_{j=1}^{k} \exp\big(\mathcal{T}\big(\mathcal{L}_j(t) - \mathcal{L}_j(t-1)\big)\big)}, \qquad i \in \{1, \ldots, k\}$$

Здесь $\mathcal{L}_i$ — значение функции потерь для слагаемого i на шаге оптимизации t, а $\mathcal{T}$ — настраиваемый гиперпараметр, определяющий веса различных компонентов функции потерь в зависимости от их динамики.

При $\mathcal{T} > 0$ алгоритм придаёт больший вес компоненту с наибольшим положительным изменением функции потерь — то есть компоненту, который обучается хуже всего. При $\mathcal{T} < 0$ больший вес получают компоненты с наиболее отрицательным изменением функции потерь — те, которые обучаются лучше всего. При $\mathcal{T} = 0$ все компоненты имеют одинаковый вес независимо от изменения их функции потерь.

Функция Softmax

Для вектора оценок (логитов) $\mathbf{z} = (z_1, \ldots, z_K)$ функция Softmax задаётся так:

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i = 1, \ldots, K.$$

Выходные значения можно интерпретировать как вероятности классов:

$$p_i = \text{softmax}(\mathbf{z})_i, \qquad p_i \geq 0, \qquad \sum_{i=1}^{K} p_i = 1.$$

Интерпретация

Softmax преобразует произвольные оценки (логиты) в распределение вероятностей по классам. Чем выше логит, тем больше вероятность соответствующего класса.

Relative Loss Balancing with Random Lookback (ReLoBRaLo) I

$$\lambda_i^{bal}(t,t') = m \cdot \frac{\exp\left(\frac{\mathcal{L}_i(t)}{\mathcal{T}\mathcal{L}_i(t')}\right)}{\sum_{j=1}^{m} \exp\left(\frac{\mathcal{L}_j(t)}{\mathcal{T}\mathcal{L}_j(t')}\right)}, \qquad i \in \{1,\dots,m\} \tag{4}$$

$$\lambda_i^{hist}(t) = \rho\,\lambda_i(t-1) + (1-\rho)\lambda_i^{bal}(t,0) \tag{5}$$

$$\lambda_i(t) = \alpha\,\lambda_i^{hist} + (1-\alpha)\lambda_i^{bal}(t,t-1) \tag{6}$$

Здесь α – эскпоненциальная скорость затухания, ρ – случайная величина Бернулли. Мат. ожидание $\mathbb{E}[\rho]$ должно быть близко к 1. Промежуточная переменная $\lambda_i^{bal}(t,t')$ расчитывает веса исходя из относительных улучшений каждого слагаемого между итерациями

Relative Loss Balancing with Random Lookback (ReLoBRaLo) II

t и t'. Следующий шаг – расчет переменной $\lambda_i^{hist}(t)$, которая определяет, рассчитываются ли веса исходя из прошлого шага ($\rho = 1$) или из расчета $\lambda_i^{bal}(t, t')$ ($\rho = 0$). Из-за этого в названии метода присутствует “random lookbacks”. В конечном итоге, вес $\lambda_i(t)$ для i-ого слагаемого рассчитывается по формуле (6), где α контролирует “значимость” весов с предыдущего шага и с текущего.

Случайная величина Бернулли

Пусть X — случайная величина Бернулли с параметром p, где $0 \leq p \leq 1$. Она принимает два значения:

$$X \in \{0, 1\}, \qquad \mathbb{P}(X = 1) = p, \qquad \mathbb{P}(X = 0) = 1 - p.$$

x	0	1
$\mathbb{P}(X = x)$	$1 - p$	p

$$\mathbb{E}[X] = p, \qquad \mathrm{Var}(X) = p(1 - p).$$

Интерпретация

$X = 1$ — успех, $\qquad X = 0$ — неуспех.

Параметры ReLoBRaLo I

$\alpha \in [0.9..0.999]$ – “память” модели
$\mathbb{E}[\rho] = 0$ – loss соответсвует начальному шагу, $\mathbb{E}[\rho] = 1$ – loss только с последнего шага. Выбирая $\mathbb{E}[\rho]$ между 0 и 1, можно ослабить значения для α.
$\mathcal{T}$ – параметр . При $\mathcal{T} \to \infty$ все значения весом будут: $\hat{\lambda}_i^{t)} = 1$. При $\mathcal{T} \to 0$ функция softmax превращается в argmax. $\hat{\lambda}_i^{(t)} = k$ для слагаемого с самым маленьким относительным весом и $\hat{\lambda}_i^{(t)} = 0$ для других.

Параметры ReLoBRaLo II

Table 1
Hyperparameters for architecture and training settings together with ranges as used for Bayesian Optimisation.

Hyperparameter	Range	Log-scaling
Learning Rate l_r	$[10^{-6}, 10^{-2}]$	Yes
Layers d_K	[2, 4]	No
Neurons per Layer w_K	[32, 512]	No
Exponential Decay Rate α	[0, 1]	No
Temperature $\mathcal{T} = 10^t$	$[10^{-6}, 10^2]$	Yes
Expected Saudade $\mathbb{E}[\rho]$	[0, 1]	No
Activation function σ	{*tanh*, *sigmoid*}	No

Рис. 2 – Гиперпараметры для ReLoBRaLo [2]

1. *Wang S.*, *Teng Y.*, *Perdikaris P.* — Understanding and mitigating gradient pathologies in physics-informed neural networks. — 2020. — Янв. — URL: http://arxiv.org/abs/2001.04536.

2. *Bischof R.*, *Kraus M. A.* — Multi-Objective Loss Balancing for Physics-Informed Deep Learning. — // Computer Methods in Applied Mechanics and Engineering. — 2025. — Май. — Т. 439. — ISSN 00457825. — DOI: 10.1016/j.cma.2025.117914.

3. GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks. — Тех. отч. / Z. Chen [и др.]. — 2018.